

TABLE OF CONTENTS

- 1 Motivation: The XOR Problem
- 2 Non-Linearity in Neural Networks
- 3 The Four Components of DFNN
- 4 Training DFNN
- 5 Making Predictions
- 6 Impact of Depth and Width
- 7 DFNN Architecture Design
- 8 Implementation Tips
- 9 Summary



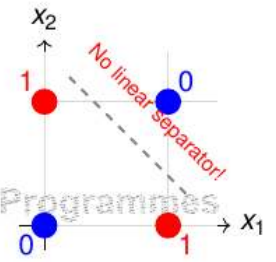
THE XOR PROBLEM

XOR (Exclusive OR): A classic non-linearly separable problem

Visual Representation:

Truth Table:

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0



Challenge: No single straight line can separate the two classes!

LINEAR MODELS HAVE LIMITATIONS

Recall: Linear models uses a single neuron with activation function $f(\cdot)$

$$\hat{y} = f(\mathbf{w}^T \mathbf{x} + b) \tag{1}$$

Problem: Many real-world relationships are **non-linear**

- Image recognition (pixel intensities $\rightarrow$ object classes)
- Natural language understanding (words $\rightarrow$ sentiment)
- Medical diagnosis (symptoms $\rightarrow$ disease probability)
- Game playing (board state $\rightarrow$ optimal move)

Linear models cannot solve non-linearly separable problems!

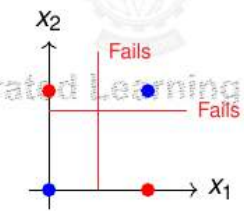
WHY SINGLE NEURON FAILS ON XOR

Single neuron model:

$$\hat{y} = \sigma(w_1 x_1 + w_2 x_2 + b) \tag{2}$$

where σ is any activation function.

Decision boundary: $w_1 x_1 + w_2 x_2 + b = 0$ (a line/hyperplane)



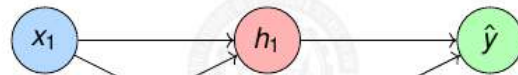
Conclusion: We need models that can learn non-linear decision boundaries!

SOLUTION: MULTI-LAYER PERCEPTRON (MLP)



Key Idea: Stack multiple layers of neurons with non-linear activations

Input Layer Hidden Layer Output Layer



Non-linear activation functions
enable non-linear boundaries

7 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

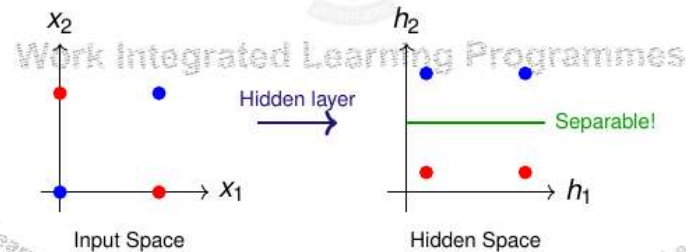
MLP SOLVES XOR



Architecture: 2 inputs → 2 hidden units → 1 output

How it works:

- Hidden layer creates new feature representations
- Each hidden unit learns a linear separator
- Output layer combines these separators
- Non-linear activation enables complex boundaries



8 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TABLE OF CONTENTS



- 1 Motivation: The XOR Problem
- 2 Non-Linearity in Neural Networks
- 3 The Four Components of DFNN
- 4 Training DFNN
- 5 Making Predictions
- 6 Impact of Depth and Width
- 7 DFNN Architecture Design
- 8 Implementation Tips
- 9 Summary

9 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

WHAT IS NON-LINEARITY?



Definition

A function f is non-linear if it does not satisfy the linearity property:

$$f(ax_1 + bx_2) \neq af(x_1) + bf(x_2) \quad (3)$$

for all scalars a, b and inputs x_1, x_2 .

Why non-linearity matters:

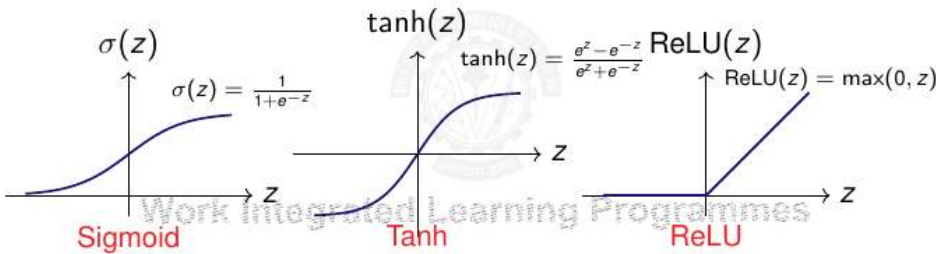
- Without non-linearity, deep networks collapse to single-layer models
- Non-linear activations enable learning complex patterns

10 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

COMMON ACTIVATION FUNCTIONS

Activation functions introduce non-linearity



ACTIVATION FUNCTION PROPERTIES

Property	Sigmoid	Tanh	ReLU	Leaky ReLU
Range	(0, 1)	(-1, 1)	$[0, \infty)$	$(-\infty, \infty)$
Zero-centered	✗	✓	✗	✓
Differentiable	✓	✓	✗ at 0	✗ at 0
Vanishing gradient	High	Moderate	No	No
Computational cost	High	High	Low	Low
Typical use	Output layer	Hidden layer	Hidden layer	Hidden layer

Modern preference: ReLU and variants for hidden layers

- Fast computation (simple thresholding)
- No vanishing gradient for positive values
- Sparse activation (biological plausibility)

WHY RELU WORKS WELL?

ReLU: $f(z) = \max(0, z)$

Advantages:

- 1 Computational efficiency
 - ▶ Simple comparison and multiplication
 - ▶ Much faster than sigmoid/tanh (no exponentials)
- 2 Avoids vanishing gradients
 - ▶ Gradient is 1 for $z > 0$, flows unchanged through network
 - ▶ Enables training of very deep networks
- 3 Sparse activation
 - ▶ Many neurons output exactly 0
 - ▶ More efficient representations
 - ▶ Closer to biological neurons

Caveat: "Dying ReLU" problem when neurons always output 0

- Solution: Use Leaky ReLU, PReLU, or ELU variants

TABLE OF CONTENTS

- 1 Motivation: The XOR Problem
- 2 Non-Linearity in Neural Networks
- 3 The Four Components of DFNN
- 4 Training DFNN
- 5 Making Predictions
- 6 Impact of Depth and Width
- 7 DFNN Architecture Design
- 8 Implementation Tips
- 9 Summary

DEEP FEED-FORWARD NEURAL NETWORK (DFNN)



Definition

A DFNN is a neural network with one or more **hidden layers** between input and output layers, where information flows forward through non-linear transformations.

Complete system requires four components:

- 1 Data: Input features and target outputs
- 2 Model: Multi-layer architecture with non-linear activations
- 3 Objective Function: Loss function to minimize
- 4 Learning Algorithm: Backpropagation with gradient descent

This is the same framework as linear models, but with added depth and non-linearity!

15 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

DATA REPRESENTATION



$$\text{Dataset: } \mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N \quad (4)$$

$$\text{Input Design Matrix: } \mathbf{X} \in \mathbb{R}^{N \times d} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \mathbf{x}^{(2)T} \\ \vdots \\ \mathbf{x}^{(N)T} \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \cdots & x_d^{(N)} \end{bmatrix} \quad (5)$$

$$\text{Target Matrix: } \mathbf{Y} \in \mathbb{R}^{N \times K} = \begin{bmatrix} \mathbf{y}^{(1)T} \\ \mathbf{y}^{(2)T} \\ \vdots \\ \mathbf{y}^{(N)T} \end{bmatrix} \quad (6)$$

where K is the output dimension (1 for regression, C for classification with C classes).

16 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

MINI-BATCH REPRESENTATION



For computational efficiency, we process data in mini-batches

$$\text{Mini-batch of size } B: \mathcal{B} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i \in \text{batch}} \quad (7)$$

$$\text{Mini-batch input matrix: } \mathbf{X}_{\mathcal{B}} \in \mathbb{R}^{B \times d} \quad (8)$$

$$\text{Mini-batch target matrix: } \mathbf{Y}_{\mathcal{B}} \in \mathbb{R}^{B \times K} \quad (9)$$

Common batch sizes: $B \in \{32, 64, 128, 256\}$

- Balances computational efficiency with gradient stability
- Enables parallelization on GPUs

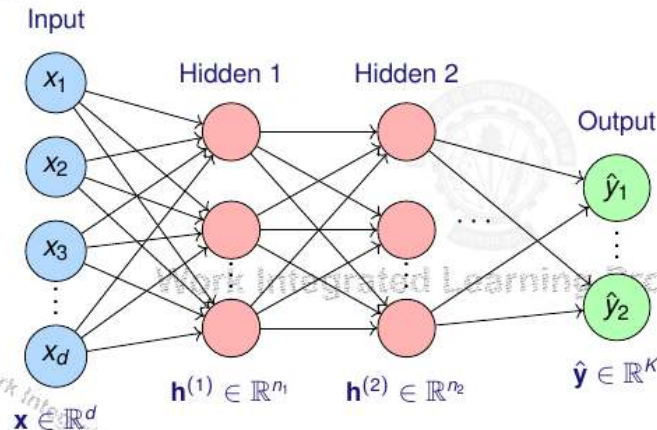
17 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

DFNN ARCHITECTURE



Network with L layers:



Notation:

- L : Number of layers (including output, excluding input)
- n_{ℓ} : Number of units in layer ℓ
- $d = n_0$: Input dimension
- $K = n_L$: Output dimension

18 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



LAYER-WISE COMPUTATION

For each layer $\ell = 1, 2, \dots, L$:

Parameters: $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}}$ (weight matrix) (10)

$\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_{\ell}}$ (bias vector) (11)

Initial condition: $\mathbf{h}^{(0)} = \mathbf{x}$ (input) (12)

Pre-activation (linear transformation): $\mathbf{z}^{(\ell)} = \mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \in \mathbb{R}^{n_{\ell}}$ (13)

Activation (non-linear transformation): $\mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) \in \mathbb{R}^{n_{\ell}}$ (14)

Final output: $\hat{\mathbf{y}} = \mathbf{h}^{(L)}$ (15)

where $\sigma^{(\ell)}$ is the activation function for layer ℓ (applied element-wise).

19 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



TOTAL PARAMETERS IN NETWORK

Parameter count for layer ℓ :

$$\text{Parameters}^{(\ell)} = n_{\ell-1} \times n_{\ell} + n_{\ell} = n_{\ell}(n_{\ell-1} + 1) \quad (20)$$

Total parameters in network:

$$\text{Total} = \sum_{\ell=1}^L n_{\ell}(n_{\ell-1} + 1) \quad (21)$$

Example: Network with 3-layer architecture $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ neurons

Layer 1: $784 \times 256 + 256 = 200,960$

Layer 2: $256 \times 128 + 128 = 32,896$

Layer 3: $128 \times 10 + 10 = 1,290$

Total: 235,146 parameters

Deep networks can have millions to billions of parameters!

21 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



VECTORIZED COMPUTATION FOR MINI-BATCH

For mini-batch $\mathbf{X}_B \in \mathbb{R}^{B \times d}$:

Initial: $\mathbf{H}^{(0)} = \mathbf{X}_B$ (16)

Layer ℓ computation: $\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_B\mathbf{b}^{(\ell)T} \in \mathbb{R}^{B \times n_{\ell}}$ (17)

$\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)}) \in \mathbb{R}^{B \times n_{\ell}}$ (18)

Final: $\hat{\mathbf{Y}}_B = \mathbf{H}^{(L)} \in \mathbb{R}^{B \times K}$ (19)

where:

- $\mathbf{H}^{(\ell)} \in \mathbb{R}^{B \times n_{\ell}}$ contains activations for all B examples
- $\mathbf{1}_B \in \mathbb{R}^{B \times 1}$ is a vector of ones (for broadcasting bias)
- Activation $\sigma^{(\ell)}$ applied element-wise to entire matrix

20 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



LOSS FUNCTIONS FOR DIFFERENT TASKS

The loss function depends on the task:

1. Regression (continuous output):

$$\text{MSE: } \ell(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2} \sum_{k=1}^K (\hat{y}_k - y_k)^2 \quad (22)$$

2. Binary Classification ($K = 1$, output in $(0, 1)$):

$$\text{BCE: } \ell(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (23)$$

3. Multi-class Classification ($K = C$ classes):

$$\text{CCE: } \ell(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^C y_k \log(\hat{y}_k) \quad (24)$$

where $\mathbf{y}$ is one-hot encoded and $\hat{\mathbf{y}}$ from softmax activation.

22 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



TOTAL LOSS OVER MINI-BATCH

Loss for single example:

$$\ell^{(i)} = \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)}) \quad (25)$$

Average loss over mini-batch $\mathcal{B}$ of size B :

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell^{(i)} = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)}) \quad (26)$$

where $\mathbf{W} = \{\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \dots, \mathbf{W}^{(L)}\}$ and $\mathbf{b} = \{\mathbf{b}^{(1)}, \mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}\}$ represent all network parameters.

Goal: Find parameters that minimize J :

$$\mathbf{W}^*, \mathbf{b}^* = \arg \min_{\mathbf{W}, \mathbf{b}} J(\mathbf{W}, \mathbf{b}) \quad (27)$$

23 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRAINING ALGORITHM: MINI-BATCH SGD



Algorithm 1: DFNN Training with Mini-Batch SGD (Part 1: Forward Pass)

Input: Dataset $\mathcal{D}$, architecture $(n_0, n_1, \dots, n_L)$, learning rate η , batch size B , epochs T

Output: Trained parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^L$

```

1 for  $\ell = 1$  to  $L$  do
2   Initialize  $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}}$  randomly;
3   Initialize  $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_{\ell}}$  to zeros;
4 for epoch  $t = 1$  to  $T$  do
5   Shuffle dataset  $\mathcal{D}$ ;
6   for each mini-batch  $\mathcal{B}$  of size  $B$  do
7     // Forward Pass
8      $\mathbf{H}^{(0)} \leftarrow \mathbf{X}_{\mathcal{B}}$ ;
9     for  $\ell = 1$  to  $L$  do
10       $\mathbf{Z}^{(\ell)} \leftarrow \mathbf{H}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{1}_B \mathbf{b}^{(\ell)T}$ ;
11       $\mathbf{H}^{(\ell)} \leftarrow \sigma^{(\ell)}(\mathbf{Z}^{(\ell)})$ ;
12     $\hat{\mathbf{Y}}_{\mathcal{B}} \leftarrow \mathbf{H}^{(L)}$ ;
13   $J \leftarrow \frac{1}{B} \sum_{i \in \mathcal{B}} \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$  // Compute Loss

```

25 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



MINI-BATCH STOCHASTIC GRADIENT DESCENT

Core idea: Update parameters using gradients from mini-batches

Update rule for layer ℓ :

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{W}^{(\ell)}} \quad (28)$$

$$\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{b}^{(\ell)}} \quad (29)$$

where $\eta > 0$ is the learning rate.

Challenge: How to compute $\frac{\partial J}{\partial \mathbf{W}^{(\ell)}}$ and $\frac{\partial J}{\partial \mathbf{b}^{(\ell)}}$ efficiently?

Solution: **Backpropagation** algorithm!

- Efficiently computes gradients using chain rule
- Key innovation that enabled deep learning

24 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRAINING ALGORITHM: MINI-BATCH SGD



Algorithm 2: DFNN Training with Mini-Batch SGD (Part 2: BackProp & Update)

```

// Backward Pass (Backpropagation)
13  $\delta^{(L)} \leftarrow \frac{\partial J}{\partial \mathbf{Z}^{(L)}}$  // Output layer gradient Depends on loss & activation
14 for  $\ell = L - 1$  to 1 do
15    $\delta^{(\ell)} \leftarrow (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)T}) \odot \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)})$  // Backpropagate through hidden layers
16 for  $\ell = 1$  to  $L$  do
17    $\frac{\partial J}{\partial \mathbf{W}^{(\ell)}} \leftarrow \frac{1}{B} \mathbf{H}^{(\ell-1)T} \delta^{(\ell)}$  // Compute parameter gradients
18    $\frac{\partial J}{\partial \mathbf{b}^{(\ell)}} \leftarrow \frac{1}{B} \mathbf{1}_B^T \delta^{(\ell)}$ ;
// Update parameters
19 for  $\ell = 1$  to  $L$  do
20    $\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{W}^{(\ell)}}$ ;
21    $\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{b}^{(\ell)}}$ ;

```

26 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



TABLE OF CONTENTS

- 1 Motivation: The XOR Problem
- 2 Non-Linearity in Neural Networks
- 3 The Four Components of DFNN
- 4 Training DFNN
- 5 Making Predictions
- 6 Impact of Depth and Width
- 7 DFNN Architecture Design
- 8 Implementation Tips
- 9 Summary

27 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

FORWARD PASS ALGORITHM



Algorithm 3: Forward Propagation

Input: Input $\mathbf{x} \in \mathbb{R}^d$, Parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^L$
Output: Prediction $\hat{\mathbf{y}} \in \mathbb{R}^K$, All activations $\{\mathbf{h}^{(\ell)}\}_{\ell=0}^L$

// Initialize with input

1 $\mathbf{h}^{(0)} \leftarrow \mathbf{x};$

// Forward through all layers

2 **for** $\ell = 1$ **to** L **do**

 // Linear transformation

3 $\mathbf{z}^{(\ell)} \leftarrow \mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)};$

 // Non-linear activation

4 $\mathbf{h}^{(\ell)} \leftarrow \sigma^{(\ell)}(\mathbf{z}^{(\ell)});$

 // Store activations for backpropagation

// Output is final layer activation

5 $\hat{\mathbf{y}} \leftarrow \mathbf{h}^{(L)};$

6 **return** $\hat{\mathbf{y}}, \{\mathbf{h}^{(\ell)}, \mathbf{z}^{(\ell)}\}_{\ell=1}^L;$

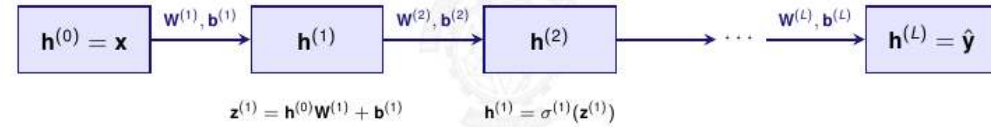
29 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

FORWARD PASS: COMPUTING PREDICTIONS



Given input $\mathbf{x} \in \mathbb{R}^d$, compute output $\hat{\mathbf{y}} \in \mathbb{R}^K$



Work Integrated Learning Programmes

28 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

FORWARD PASS: VECTORIZED FOR MINI-BATCH



For mini-batch $\mathbf{X}_B \in \mathbb{R}^{B \times d}$:

Initialize: $\mathbf{H}^{(0)} = \mathbf{X}_B \in \mathbb{R}^{B \times d}$ (30)

For each layer $\ell = 1, \dots, L$:

$\mathbf{z}^{(\ell)} = \mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_B\mathbf{b}^{(\ell)T} \in \mathbb{R}^{B \times n_\ell}$ (31)

$\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) \in \mathbb{R}^{B \times n_\ell}$ (32)

Output predictions:

$\hat{\mathbf{Y}}_B = \mathbf{H}^{(L)} \in \mathbb{R}^{B \times K}$ (33)

Computational complexity : $O(B \sum_{\ell=1}^L n_{\ell-1} n_\ell)$ per forward pass (34)

30 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



LOSS COMPUTATION

After forward pass, compute loss for the mini-batch:

$$\text{Per-example loss: } \ell^{(i)} = \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)}) \quad (35)$$

$$\text{Average loss over mini-batch: } J = \frac{1}{B} \sum_{i=1}^B \ell^{(i)} \quad (36)$$

Common loss functions:

$$\text{Regression: } MSE = \frac{1}{2} \|\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}\|^2$$

$$\text{Binary Classification: } BCE = -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

$$\text{Multi-class: } CCE = - \sum_{k=1}^C y_k^{(i)} \log(\hat{y}_k^{(i)})$$

This loss quantifies how wrong our predictions are.

31 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



BACKPROPAGATION: THE KEY ALGORITHM

Purpose

Backpropagation efficiently computes gradients of the loss w.r.t. all parameters by applying the chain rule backward through the network.

Key insight: Use chain rule recursively

Notation: Define error signal for layer ℓ :

$$\delta^{(\ell)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} \in \mathbb{R}^{B \times n_\ell} \quad (39)$$

Two main steps:

1. Backpropagate error signals: $\delta^{(L)} \rightarrow \delta^{(L-1)} \rightarrow \dots \rightarrow \delta^{(1)}$
2. Compute parameter gradients using error signals

33 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

OUTPUT LAYER GRADIENT

First step of backpropagation: compute gradient w.r.t. output layer pre-activation

Output layer gradient:

$$\delta^{(L)} = \frac{\partial J}{\partial \mathbf{Z}^{(L)}} \in \mathbb{R}^{B \times K} \quad (37)$$

For common loss-activation pairs: (for all 3, the output gradient is same.)

1. MSE + Identity: (Regression)
2. BCE + Sigmoid: (Binary classification)
3. CCE + Softmax: (Multi-class classification)

$$\delta^{(L)} = \frac{1}{B} (\hat{\mathbf{Y}} - \mathbf{Y}) \quad (38)$$

Notice the elegant form when loss and activation are paired correctly!

32 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



BACKPROPAGATING ERROR SIGNALS

$$\text{Given: } \delta^{(\ell+1)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell+1)}} \quad \text{Compute: } \delta^{(\ell)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}}$$

$$\text{Chain rule: } \delta^{(\ell)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{H}^{(\ell)}} \frac{\partial \mathbf{H}^{(\ell)}}{\partial \mathbf{Z}^{(\ell)}} \quad (40)$$

$$\text{Derivation: } \frac{\partial J}{\partial \mathbf{H}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell+1)}} \frac{\partial \mathbf{Z}^{(\ell+1)}}{\partial \mathbf{H}^{(\ell)}} = \delta^{(\ell+1)} \mathbf{W}^{(\ell+1)T} \quad (41)$$

$$\frac{\partial \mathbf{H}^{(\ell)}}{\partial \mathbf{Z}^{(\ell)}} = \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)}) \quad (\text{element-wise derivative}) \quad (42)$$

Backpropagation equation:

$$\delta^{(\ell)} = (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)T}) \odot \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)}) \quad (43)$$

where $\odot$ denotes element-wise multiplication (Hadamard product).

34 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



Given error signal $\delta^{(\ell)}$, compute gradients w.r.t. parameters:

$$\text{Weight gradient: } \frac{\partial J}{\partial \mathbf{W}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} \frac{\partial \mathbf{Z}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}} = \frac{1}{B} \mathbf{H}^{(\ell-1)T} \delta^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}} \quad (44)$$

$$\text{Bias gradient: } \frac{\partial J}{\partial \mathbf{b}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} \frac{\partial \mathbf{Z}^{(\ell)}}{\partial \mathbf{b}^{(\ell)}} = \frac{1}{B} \mathbf{1}_B^T \delta^{(\ell)} \in \mathbb{R}^{n_{\ell}} \quad (45)$$

where $\mathbf{1}_B \in \mathbb{R}^{B \times 1}$ is a vector of ones.

Key: Error signal $\delta^{(\ell)}$ is shared for both weight and bias gradients!

35 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



Algorithm 4: Backpropagation

Input: Loss J , Activations $\{\mathbf{H}^{(\ell)}, \mathbf{Z}^{(\ell)}\}_{\ell=1}^L$, Weights $\{\mathbf{W}^{(\ell)}\}_{\ell=1}^L$

Output: Gradients $\{\frac{\partial J}{\partial \mathbf{W}^{(\ell)}}, \frac{\partial J}{\partial \mathbf{b}^{(\ell)}}\}_{\ell=1}^L$

```
// Compute output layer error signal
1  $\delta^{(L)} \leftarrow \frac{\partial J}{\partial \mathbf{Z}^{(L)}}$  // Depends on loss function
// Backpropagate error signals
2 for  $\ell = L - 1$  to 1 do
3    $\delta^{(\ell)} \leftarrow (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)T}) \odot \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)})$ ;
// Compute parameter gradients
4 for  $\ell = 1$  to  $L$  do
5    $\frac{\partial J}{\partial \mathbf{W}^{(\ell)}} \leftarrow \frac{1}{B} \mathbf{H}^{(\ell-1)T} \delta^{(\ell)}$ ;
6    $\frac{\partial J}{\partial \mathbf{b}^{(\ell)}} \leftarrow \frac{1}{B} \mathbf{1}_B^T \delta^{(\ell)}$ ;
7 return  $\{\frac{\partial J}{\partial \mathbf{W}^{(\ell)}}, \frac{\partial J}{\partial \mathbf{b}^{(\ell)}}\}_{\ell=1}^L$ ;
```

36 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

ACTIVATION FUNCTION DERIVATIVES



Common activation derivatives (for backpropagation):

1. Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (46)$$

2. Tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad \tanh'(z) = 1 - \tanh^2(z) \quad (47)$$

3. ReLU:

$$\text{ReLU}(z) = \max(0, z) \quad \text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \quad (48)$$

37 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

COMPUTATIONAL EFFICIENCY OF BACKPROPAGATION



Why backpropagation is efficient:

Naïve approach: Compute each $\frac{\partial J}{\partial \mathbf{W}_{ij}^{(\ell)}}$ independently

- Requires one forward pass per parameter
- Complexity: $O(P \times C_{\text{forward}})$ where P = total parameters
- For millions of parameters, this is intractable!

Backpropagation: Reuses intermediate computations

- One forward pass + one backward pass
- Complexity: $O(C_{\text{forward}} + C_{\text{backward}}) \approx 2 \times O(C_{\text{forward}})$
- Backward pass has same complexity as forward pass

Speedup: $\frac{P}{2}$ (e.g., $\sim 100,000 \times$ for $P = 200,000$)

Key insight: Dynamic programming via chain rule!

38 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



PARAMETER UPDATE STEP

After computing gradients, update all parameters:

For each layer $\ell = 1, \dots, L$: (Vector form for single example)

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \mathbf{h}^{(\ell-1)T} \delta^{(\ell)} \quad (49)$$

$$\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \delta^{(\ell)} \quad (50)$$

where $\eta > 0$ is the learning rate.

This moves parameters in the direction that reduces the loss!

39 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



USING TRAINED DFNN FOR PREDICTION

After training, prediction is simply forward propagation:

Input: New example $\mathbf{x}_{\text{new}} \in \mathbb{R}^d$

Process:

1 Initialize: $\mathbf{h}^{(0)} = \mathbf{x}_{\text{new}}$

2 For $\ell = 1$ to L :

$$\mathbf{z}^{(\ell)} = \mathbf{h}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \quad (51)$$

$$\mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) \quad (52)$$

3 Output: $\hat{\mathbf{y}}_{\text{new}} = \mathbf{h}^{(L)}$

Interpretation depends on task:

- Regression: $\hat{\mathbf{y}}_{\text{new}}$ is the predicted continuous value
- Binary classification: $\hat{\mathbf{y}}_{\text{new}} > 0.5 \rightarrow$ class 1, else class 0
- Multi-class: $\arg \max_k \hat{\mathbf{y}}_{k,\text{new}}$ is the predicted class

41 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TABLE OF CONTENTS

- 1 Method: The XOR Problem
- 2 Non-Linearity in Neural Networks
- 3 The Four Components of DFNN
- 4 Training DFNN
- 5 Making Predictions
- 6 Impact of Depth and Width
- 7 DFNN Architecture Design
- 8 Implementation Tips
- 9 Summary

40 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



BATCH PREDICTION

Efficiently predict for multiple examples:

Input: Test set $\mathbf{X}_{\text{test}} \in \mathbb{R}^{N_{\text{test}} \times d}$

Vectorized forward pass:

$$\mathbf{H}^{(0)} = \mathbf{X}_{\text{test}} \quad (53)$$

$$\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{1}_{N_{\text{test}}} \mathbf{b}^{(\ell)T} \quad (54)$$

$$\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)}) \quad (55)$$

$$\hat{\mathbf{Y}}_{\text{test}} = \mathbf{H}^{(L)} \in \mathbb{R}^{N_{\text{test}} \times K} \quad (56)$$

Advantages:

- Efficient matrix operations (GPU acceleration)
- All predictions in one forward pass
- Essential for large-scale deployment

42 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TABLE OF CONTENTS

1. Motivation: The XOR Problem
2. Non-Linearity in Neural Networks
3. The Four Components of DFNN
4. Training DFNN
5. Making Predictions
6. Impact of Depth and Width
7. DFNN Architecture Design
8. Implementation Tips
9. Summary

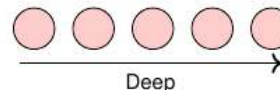
43 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

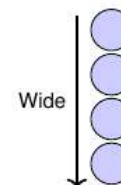
NETWORK DEPTH VS WIDTH

Two ways to increase model capacity:

Depth (more layers):



Width (more units per layer):



Properties:

- Hierarchical features
- More expressive
- Better generalization

Properties:

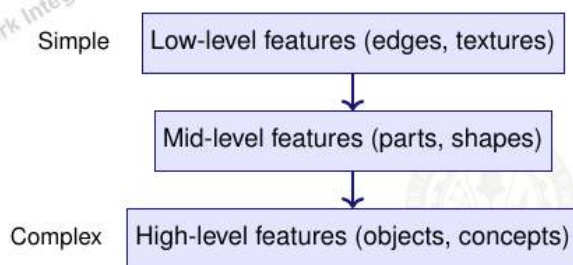
- More parallel features
- Higher capacity
- Risk of overfitting

44 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

WHY DEPTH MATTERS?

Deep networks learn hierarchical representations:



Deep networks can represent exponentially more functions than shallow ones with the same number of parameters!

Example: Image classification

- Layer 1: Detect edges and colors
- Layer 2: Combine into textures and simple shapes
- Layer 3: Recognize object parts (eyes, wheels)
- Layer 4+: Identify complete objects (faces, cars)

45 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

UNIVERSAL APPROXIMATION THEOREM

Theorem (Cybenko, 1989; Hornik et al., 1989)

A feed-forward network with a single hidden layer containing a finite number of neurons can approximate any continuous function on a compact subset of $\mathbb{R}^n$, given an appropriate activation function.

Depth enables efficient learning of hierarchical structure!

What this means:

- Theoretically, even shallow networks are powerful
- **BUT:** May require exponentially many hidden units
- Deep networks are much more parameter-efficient

Practice:

- Deep networks learn better representations
- Achieve better performance with fewer parameters
- Generalize better to new data

46 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

CHALLENGES WITH VERY DEEP NETWORKS

Problems that arise with increasing depth:

1. Vanishing Gradients:

- Gradients become exponentially small in early layers
- Network fails to learn useful features
- Solution: ReLU activations, careful initialization, normalization

2. Exploding Gradients:

- Gradients become exponentially large
- Causes numerical instability (NaN/Inf values)
- Solution: Gradient clipping, proper initialization

3. Degradation Problem:

- Very deep networks harder to optimize
- Training error may increase with depth
- Solution: Residual connections (ResNets), skip connections

47 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

DESIGN CONSIDERATIONS

Key decisions when designing a DFNN:

1. Number of layers L
 - ▶ How deep should the network be?
2. Units per layer $n_1, n_2, \dots, n_L$
 - ▶ How wide should each layer be?
3. Activation functions $\sigma^{(\ell)}$
 - ▶ Which non-linearity for each layer?
4. Output layer design
 - ▶ Depends on task (regression vs classification)
5. Regularization
 - ▶ How to prevent overfitting?

No universal answer - depends on problem, data, and computational resources!

49 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TABLE OF CONTENTS

1. Motivation: The XOR Problem

2. Non-Linearity in Neural Networks

3. The Four Components of DFNN

4. Training DFNN

5. Making Predictions

6. Impact of Depth and Width

7. DFNN Architecture Design

8. Implementation Tips

9. Summary

48 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

OUTPUT LAYER DESIGN

Output layer depends on the task:

1. Regression (continuous values):

- Units: K = dimension of output
- Activation: Identity (linear)
- Loss: Mean Squared Error (MSE)

2. Binary Classification (2 classes):

- Units: $K = 1$
- Activation: Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
- Loss: Binary Cross-Entropy (BCE)

3. Multi-class Classification (C classes):

- Units: $K = C$
- Activation: Softmax $\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}$
- Loss: Categorical Cross-Entropy (CCE)

50 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

GENERAL ARCHITECTURE GUIDELINES

1. Start Simple:

- Begin with 2-3 hidden layers
- 10-100 units per layer (depending on problem scale)
- Gradually increase complexity if needed

2. Common Patterns:

- Decreasing width: $n_1 > n_2 > \dots > n_L$
 - ▶ Funnel architecture (compression)
- Constant width: $n_1 = n_2 = \dots = n_{L-1}$
 - ▶ Uniform representation
- Hourglass: Wide $\rightarrow$ Narrow $\rightarrow$ Wide
 - ▶ Learns compressed representations

3. Scale with Data:

- More data $\rightarrow$ can support larger networks
- Less data $\rightarrow$ keep networks smaller to avoid overfitting

51 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

REGULARIZATION TECHNIQUES

Prevent overfitting with:

1. L2 Regularization (Weight Decay):
2. Dropout:
3. Early Stopping:
4. Batch Normalization:

Note: You will learn about this in Module 10 and 11

53 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

HIDDEN LAYER ACTIVATION FUNCTIONS

Activation	When to Use	Avoid When
ReLU	Default choice Fast, no vanishing gradient	Very deep networks (dying ReLU)
Leaky ReLU	Deep networks Fixes dying ReLU	Never (safe choice)
Tanh	Small networks Zero-centered	Very deep networks (vanishing gradient)
Sigmoid	Avoid for hidden layers (only for binary output)	Hidden layers (strong saturation)

Modern recommendation:

- Default: ReLU for hidden layers
- Alternative: Leaky ReLU, ELU for very deep networks
- Avoid: Sigmoid/Tanh in hidden layers (unless specific reason)

52 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TABLE OF CONTENTS

1. Motivation: The XOR Problem
2. Non-Linearity in Neural Networks
3. The Four Components of DFNN
4. Training DFNN
5. Making Predictions
6. Impact of Depth and Width
7. DFNN Architecture Design
8. Implementation Tips
9. Summary

54 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



PARAMETER INITIALIZATION

Proper initialization is critical for training success!

✗ Bad: Zero initialization

✓ Good: Random initialization

- Xavier/Glorot Initialization for Sigmoid / Tanh
- He Initialization for ReLU
- Biases: Initialize to zero: $\mathbf{b}^{(\ell)} = \mathbf{0}$

Note: You will learn about this in Module 10 and 11

55 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



FEATURE PREPROCESSING

Preprocess features before training:

1. Normalization/Standardization:

- Standardization (recommended)
- Min-Max Scaling
- Apply same transformation to validation/test sets

2. Handle Missing Values:

- Impute with mean/median/mode
- Or use indicator variables

3. Encode Categorical Variables:

- One-hot encoding for nominal categories
- Ordinal encoding for ordered categories

57 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TRAINING HYPERPARAMETERS TO TUNE

1. Learning Rate η :

- Start with: $\eta \in \{0.001, 0.01, 0.1\}$
- Consider adaptive methods (Adam with $\eta = 0.001$)

2. Batch Size B :

- Common: $B \in \{32, 64, 128, 256\}$
- Larger batch $\rightarrow$ faster but may hurt generalization
- Smaller batch $\rightarrow$ better generalization but slower
- Limited by GPU memory

3. Number of Epochs T :

- Use early stopping based on validation loss
- Typically 50-200 epochs for moderate datasets
- Monitor for overfitting

56 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956



MONITORING TRAINING PROGRESS

Track metrics during training:



What to monitor:

- Training loss (should decrease steadily)
- Validation loss (should decrease then plateau/increase)
- Training/validation accuracy (for classification)
- Gradient norms (detect vanishing/exploding gradients)

58 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

DIAGNOSING TRAINING ISSUES

Symptom	Diagnosis	Solution
Loss = NaN/Inf	Numerical instability	Decrease η , check gradients, normalize inputs
Loss not decreasing	Learning rate too small OR bug	Increase η , verify gradient computation
Loss oscillating	Learning rate too large	Decrease η , use learning rate schedule
Training loss $\downarrow$, validation loss $\uparrow$	Overfitting	Add regularization, dropout, more data
Both losses high	Underfitting	Increase capacity (wider/deeper), train longer
Slow convergence	Poor initialization OR bad features	Use proper init (He/Xavier), normalize features
Gradients $\rightarrow 0$	Vanishing gradients	Use ReLU, better init, batch normalization
Gradients $\rightarrow \infty$	Exploding gradients	Gradient clipping, decrease η

59 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

DEBUGGING CHECKLIST

Before training:

- Features normalized/standardized
- Data split into train/val/test
- Output layer matches task (sigmoid for binary, softmax for multi-class)
- Loss function matches output activation
- Weights initialized properly (not all zeros!)

During training:

- Loss decreasing on training set
- Validation loss tracked separately
- Gradients neither vanishing nor exploding
- No NaN/Inf values in loss or activations
- Training accuracy improving

Sanity checks:

- Try overfitting a small batch (should reach 0 loss)
- Verify shapes of all matrices/tensors
- Test forward pass before adding backprop

60 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

IMPLEMENTATION BEST PRACTICES

1. Vectorization:

- Always use matrix operations, never loops over examples
- Enables GPU acceleration

2. Modular Code:

- Separate forward pass, loss, backward pass functions
- Test each component independently

3. Numerical Stability:

- Use stable implementations (e.g., log-sum-exp for softmax)
- Clip gradients if needed: $\|\nabla\| > \text{threshold}$
- Monitor for NaN/Inf values

4. Reproducibility:

- Set random seeds
- Save hyperparameters
- Track experiments systematically

61 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

TABLE OF CONTENTS

1 Method: The XOR Problem

2 Non-Linearity in Neural Networks

3 The Four Components of DFNN

4 Training DFNN

5 Making Predictions

6 Impact of Depth and Width

7 DFNN Architecture Design

8 Implementation Tips

9 Summary

62 / 67

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956